20262819《Linux内核原理与分析》第三周作业

> **本周主题｜阅读 Linux 内核源码，理解简单多道程序内核的运行机制** 阅读范围：《庖丁解牛Linux操作系统分析》第3章“Linux内核源代码及调试环境”，书内 pp.90–111（PDF pp.109–130）。配套实验：基于 Linux 3.9.4 的 mykernel 时间片轮转多道程序内核。

基本信息	内容
姓名	刘顺文
学号	20262819
作业周次	第三周
整理日期	2026-10-11
实验平台	蓝桥云课 Linux 实验环境，QEMU 运行实验内核
实验源码版本	Linux 3.9.4，源码目录为 `~/LinuxKernel/linux-3.9.4`

01 | 本周学习目标与进度

本周围绕“怎样从源码理解内核执行过程”展开。教材第三章介绍源码组织、编译配置和调试方法；配套实验则在已有内核框架中增加任务描述、初始化和调度逻辑，观察多个任务的运行输出。二者分别对应完整 Linux 内核和精简教学内核，需要结合理解，也需要区分版本与实现。

学习任务	完成情况	本周记录
阅读教材第3章	已完成	覆盖 §3.1、§3.2、§3.3及章末实验说明，独立整理阅读总结
云班课视频	已完成	视频已观看
默认 mykernel 编译与运行	已有记录	补丁、配置、bzImage 生成和主循环 / 时钟输出
增加多任务代码并重新编译	已有记录	三个源码文件的编辑截图、文件检查和编译完成截图
观察多任务输出	已有记录	QEMU 中分别观察到 process 0、process 1

02 | 教材阅读与理解

2.1 本章主线：从定位源码到跟踪执行

第三章主要讨论如何阅读和调试 Linux 内核。其学习顺序是：先了解源码目录及关键入口，再配置源码阅读工具，接着编译内核、准备内存根文件系统，最后通过 QEMU 与 GDB 跟踪启动过程。这样可以把静态代码中的函数关系与运行时的调用栈、断点和进程创建联系起来。

教材以 Linux 5.4.34 和 x86-64 平台为主要示例。本周 mykernel 实验使用 Linux 3.9.4 的32位 x86 教学实现，因此教材中的配置、函数细节与实验代码不能直接混用。

2.2 源码目录与内核入口

内核源码体量很大，阅读时应围绕目标模块寻找相关文件和调用关系。教材指出，驱动和体系结构相关代码占比较大，初学时可先把注意力放在初始化、调度和内存管理等核心部分。

目录	主要作用	阅读时的关注点
`arch/`	与 CPU 架构相关的代码	x86、ARM64 的启动和底层实现不同
`init/`	内核启动初始化	`init/main.c` 中的 `start_kernel()`
`kernel/`	核心内核代码	调度等核心机制
`mm/`	内存管理	内存管理模块的初始化和运行
`fs/`、`ipc/`、`net/`	文件系统、进程间通信、网络	根据具体问题继续定位
`drivers/`	设备驱动	内核与具体硬件之间的联系
`include/`、`lib/`	公共头文件、内核公共函数	类型定义、接口与内核内部工具函数

普通 C 程序常从 `main()` 开始分析，而 Linux 内核的重要 C 初始化入口是 `start_kernel()`。在此之前还有体系结构相关的汇编代码，负责为后续 C 代码准备运行环境。因此，“从 start_kernel 开始阅读”不等于“CPU 上电后的第一条指令就是 start_kernel”。

内核代码使用自己的接口，例如实验中的 `printk()` 用于输出内核信息。理解函数时应查实际内核版本的定义，不能直接按用户态标准库函数理解。

2.3 配置源码阅读工具与内核构建

教材使用 VS Code 阅读源码。由于内核存在大量条件编译和架构专用头文件，单纯设置几个头文件目录不足以准确反映实际构建。书中通过生成 `compile_commands.json`，结合 C/C++ 插件和相关配置，改善跳转、宏和类型识别；还通过排除无关架构与生成文件减少搜索范围。

我的理解是：源码阅读环境需要与正在分析的内核配置相匹配。编辑器给出的提示可以帮助定位代码，但实际编译结果仍以构建系统为准。

内核构建可概括为“源码 → 配置 → 编译 → 镜像”。`.config` 决定启用哪些功能，`make` 根据配置编译。教材列出的常见配置方式包括：

配置方式	作用
`make menuconfig`	通过文本菜单配置内核选项
`make defconfig`	使用目标平台提供的默认配置
`make allnoconfig`	尽量关闭可选功能，形成精简配置
`make allyesconfig`	尽量启用可选功能

补充说明：配置项仍受依赖关系和必要选项约束，“allnoconfig”不能理解为生成一个完全没有功能的内核。本周实验采用 `make allnoconfig`，是为了在精简配置下突出 mykernel 的教学逻辑。

2.4 内存根文件系统与 QEMU/GDB 调试

教材使用 BusyBox 构建精简的内存根文件系统。BusyBox 提供基本用户态工具，静态编译可以减少对目标环境中动态库的依赖。准备必要目录、设备节点和可执行的 `/init` 脚本后，再打包为 `rootfs.cpio.gz`，交给 QEMU 加载。

这里需要区分三个文件：

文件	用途
`arch/x86/boot/bzImage`	启动内核所用的镜像
`vmlinux`	GDB 加载符号和调试信息所用的内核文件
`rootfs.cpio.gz`	内存根文件系统，提供启动后的用户态环境

教材指出，只启动普通 Linux 内核而没有可用的根文件系统，可能在后续阶段出现 kernel panic。这个现象不等于前面的编译一定失败。本周 mykernel 将执行引向教学任务循环，不能把它的输出直接等同于完整 Linux 系统已进入用户态。

教材中 QEMU 调试参数的大小写有明确含义：`-s` 开启默认 TCP 1234 端口的 GDB 调试服务；`-S` 在启动时暂停 CPU，等待调试器继续执行。GDB 先加载 `vmlinux`，连接 QEMU，再在关键函数处设断点。

下面是对教材方法的简化整理，属于阅读笔记，并非本次实验已经执行的记录：

```
file vmlinux
target remote localhost:1234
break start_kernel
continue
backtrace
list
```

`backtrace` 用于查看调用栈，`list` 用于查看附近源码。教材还给出 VS Code 的 `tasks.json` 与 `launch.json` 配置：前者组织构建和 QEMU 启动任务，后者指定调试程序、GDB 模式、目标架构和远程地址。

教材要求启用必要的调试选项，并在其调试示例中关闭 KASLR。补充说明：KASLR 会使运行时地址发生变化，因此调试符号与实际地址的对应需要额外处理；本次截图没有 GDB 记录，不将教材中的调试步骤记为本人已完成实验。

2.5 启动过程与0、1、2号进程

本章后半部分通过 `start_kernel()`、`rest_init()`、`kernel_init()` 和 `kthreadd()`，解释内核怎样从初始化进入正常运行。

```
体系结构相关的早期启动代码
    ↓
start_kernel(): 初始化内存、调度等主要模块
    ↓
arch_call_rest_init() → rest_init()
    ├─ 创建 PID 1: kernel_init → 加载 init 用户程序
    ├─ 创建 PID 2: kthreadd → 管理内核线程创建请求
    └─ PID 0: 进入空闲执行路径
```

0号进程的描述符 `init_task` 在源码中静态初始化，不是通过通常的复制创建过程得到的。`rest_init()` 创建1号和2号任务：1号任务继续初始化并执行用户态 init 程序；2号任务 `kthreadd` 留在内核态，为其他内核线程的创建提供服务。0号任务在初始化完成后进入空闲路径。

`kernel_init()` 会尝试执行指定或默认的 init 程序，例如 `/sbin/init`、`/etc/init`、`/bin/init`、`/bin/sh`；找不到可运行的 init 时会报告错误。`kthreadd()` 则在循环中等待和处理创建请求，这也说明内核里的长期循环本身并不代表程序出错。

2.6 与本周实验的联系

第三章给出了分析完整内核的方法，本周实验把其中的任务描述、启动和切换缩小为容易观察的教学模型。`mypcb.h` 的 PCB 可以帮助理解进程描述符；`mymain.c` 的初始化体现入口、栈和任务组织；QEMU 输出则把代码的运行结果显示出来。

需要区分：实验中的 `task[0]`、`task[1]` 是 mykernel 自己定义的教学任务，与完整 Linux 的 idle、init、kthreadd 并非同一组进程。实验中的四个 PCB 也不能代替真实 Linux 的 `task_struct` 和完整调度器。

03 | 实验二：简单时间片轮转多道程序内核

3.1 实验目的与环境

实验首先运行默认 mykernel，观察主循环和时钟中断；随后加入 PCB、任务初始化及调度逻辑，重新编译，观察不同 PID 的运行输出。重点理解任务能继续执行所需的状态，以及时钟请求、任务选择和上下文切换的关系。

环境项目	由记录确认的信息
平台	蓝桥云课；截图中的用户名为 `shiyanlou`
源码	Linux 3.9.4，目录 `~/LinuxKernel/linux-3.9.4`
实验代码目录	`mykernel/`
编辑器	Vim
运行工具	QEMU
实验目标	32位 x86 教学代码

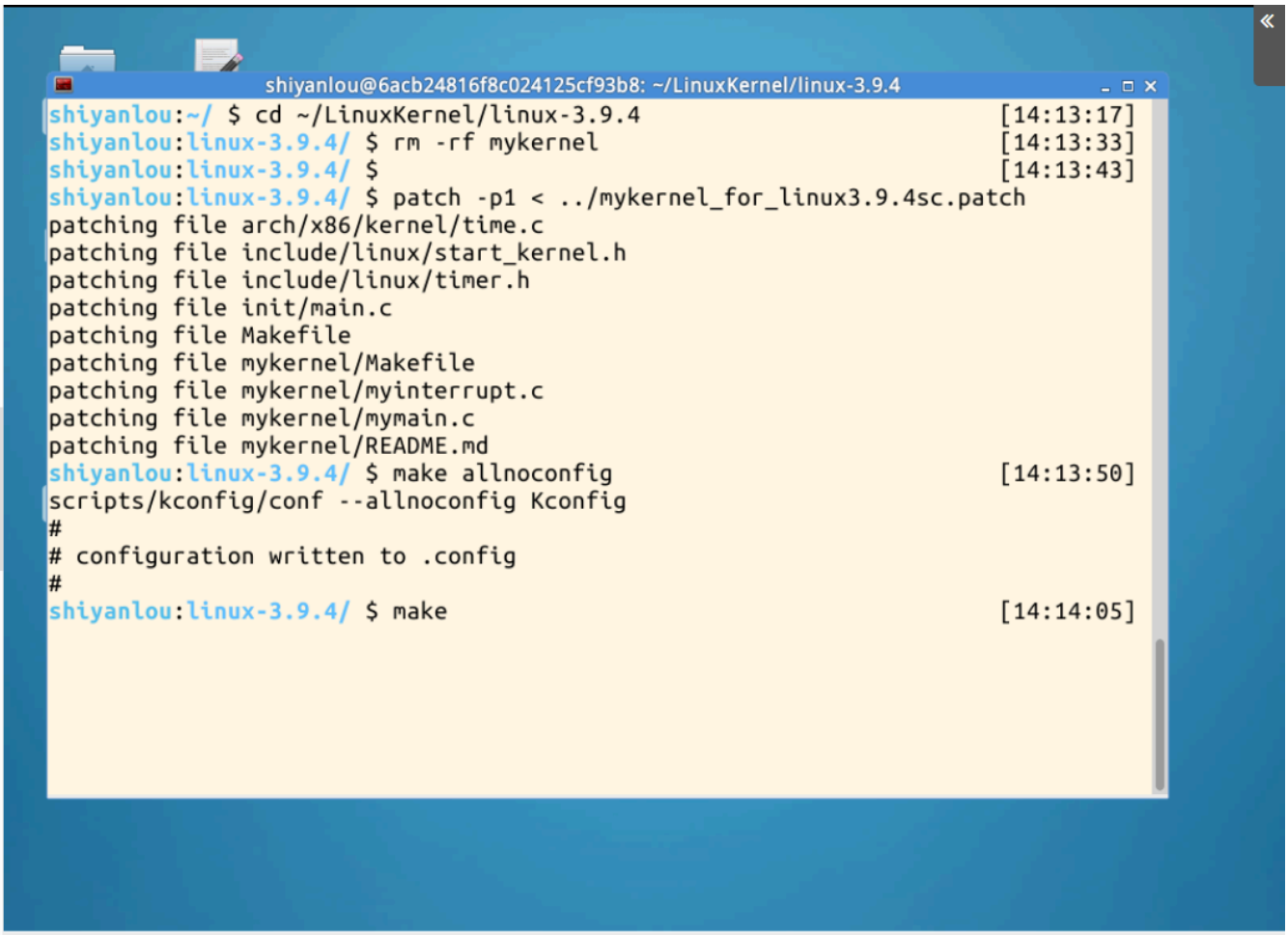
前后截图的终端主机标识不同，因此不能认定所有图片来自同一次实验会话。以下按“默认版本 → 修改代码 → 重新编译 → 运行输出”的实验阶段整理。

3.2 应用补丁、配置并开始编译

在源码根目录执行的命令记录如下：

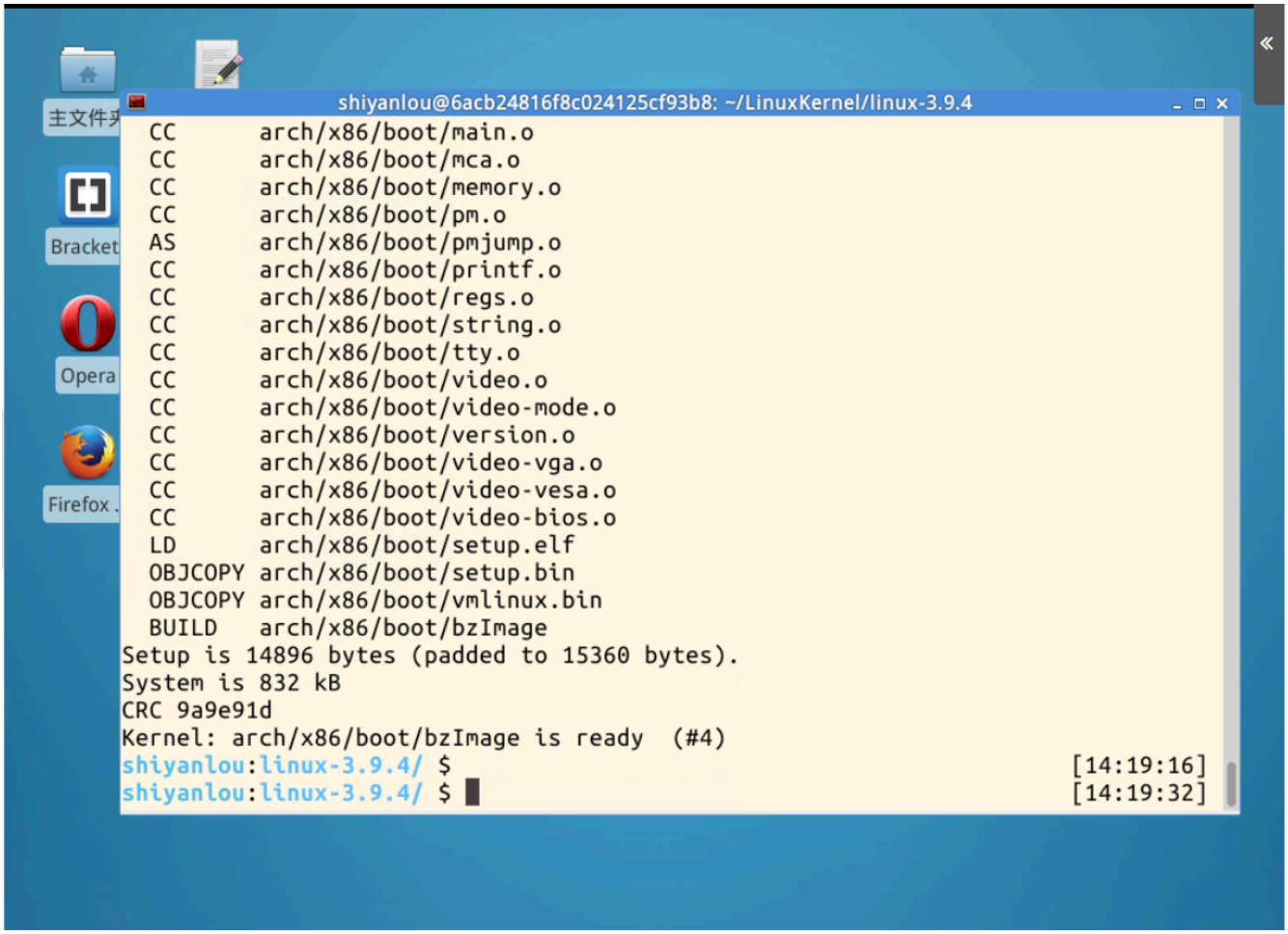
```
cd ~/LinuxKernel/linux-3.9.4
rm -rf mykernel
patch -p1 < ../mykernel_for_linux3.9.4sc.patch
make allnoconfig
make
```

这是截图中的实际操作整理。`rm -rf mykernel` 删除当前源码树中的 mykernel 子目录；它不是删除整个内核源码，也不等于清理全部旧构建文件。重复实验时应先备份该目录中的自定义代码，并检查是否已经应用补丁。



观察与分析：图1显示补丁修改了 `init/main.c`、相关头文件、Makefile 和 mykernel 源码，随后生成 `.config`。这一步将教学入口与时钟处理代码接入内核构建及执行路径；截图中未出现补丁失败提示。

3.3 默认版本编译完成

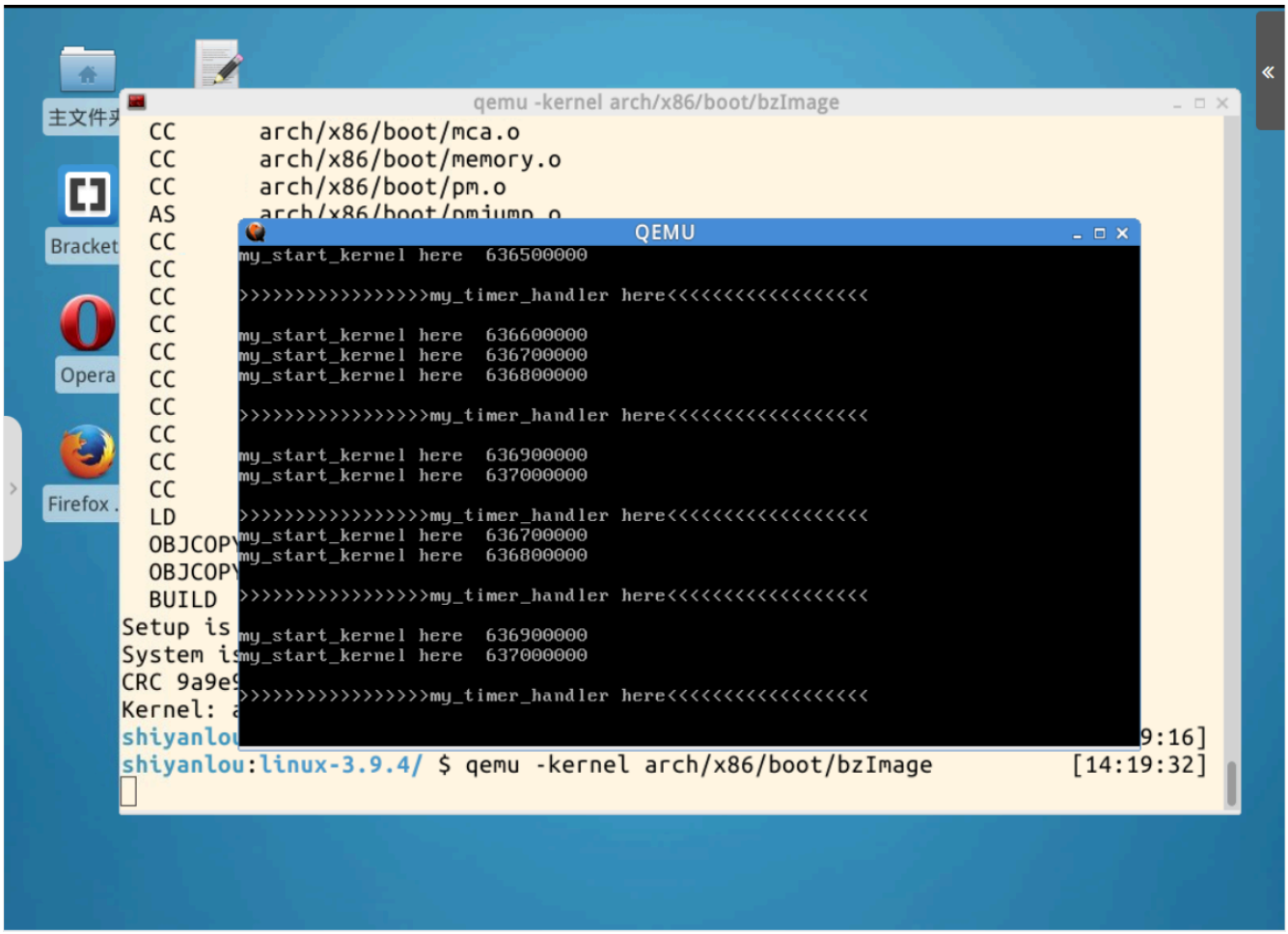


观察与分析：编译结束出现 `Kernel: arch/x86/boot/bzImage is ready (#4)`，终端返回提示符。截图还记录了 `System is 832 kB`。该提示表明内核镜像构建已经完成，下一步需要启动镜像观察其实际行为。

3.4 在 QEMU 中运行默认代码

源码根目录中的启动命令为：

```
qemu -kernel arch/x86/boot/bzImage
```

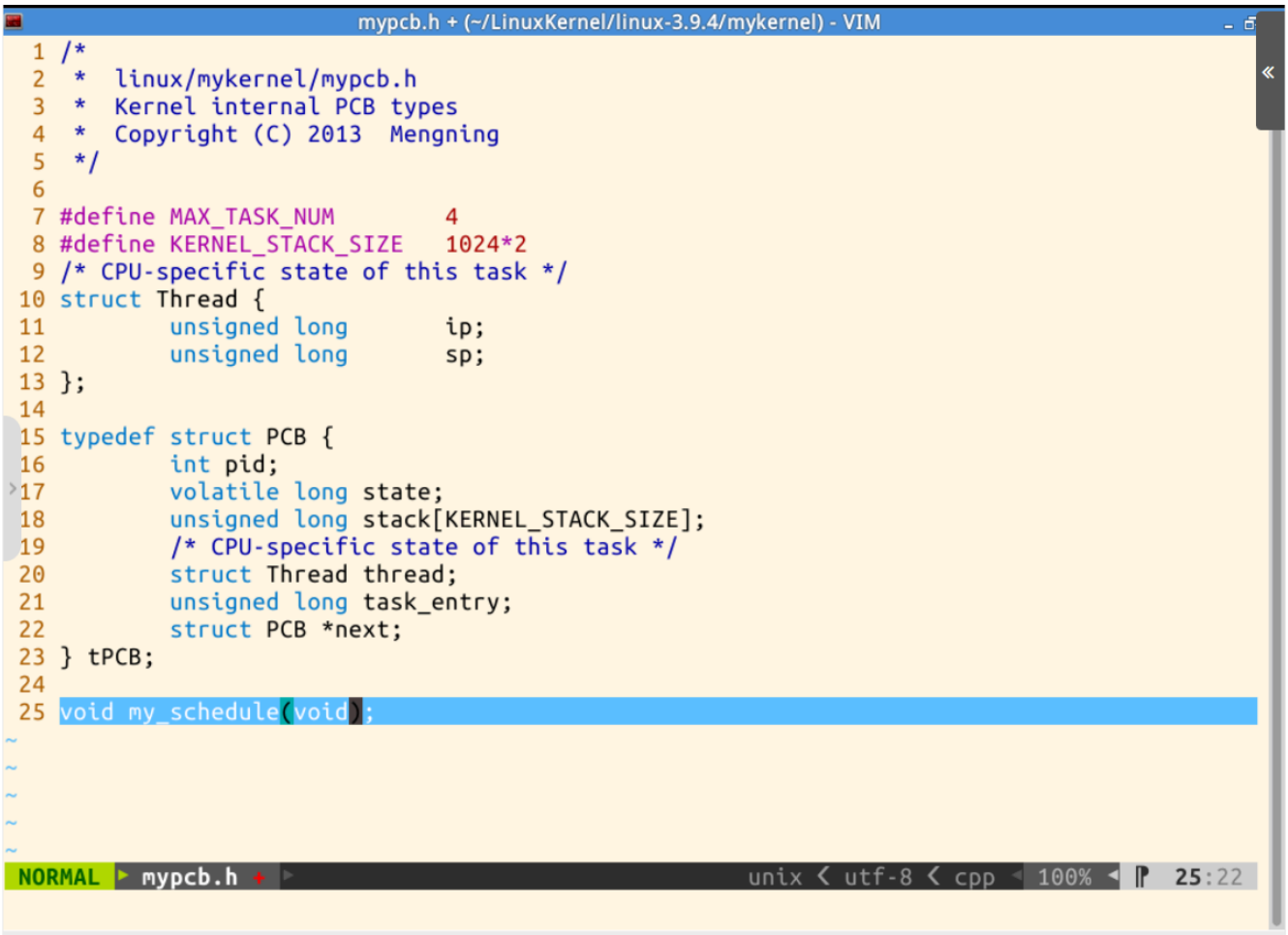


观察与分析：QEMU 窗口持续输出 `my_start_kernel here`，同时穿插 `my_timer_handler here`。这说明主循环正在执行，时钟处理函数也会周期性获得执行机会。

默认代码使用持续循环展示内核执行过程，并非仅有时钟处理函数在运行。当前缺少的是多个任务的管理和切换逻辑，所以后续需要增加 PCB、独立栈以及调度函数。

3.5 创建 mypcb.h：定义任务状态

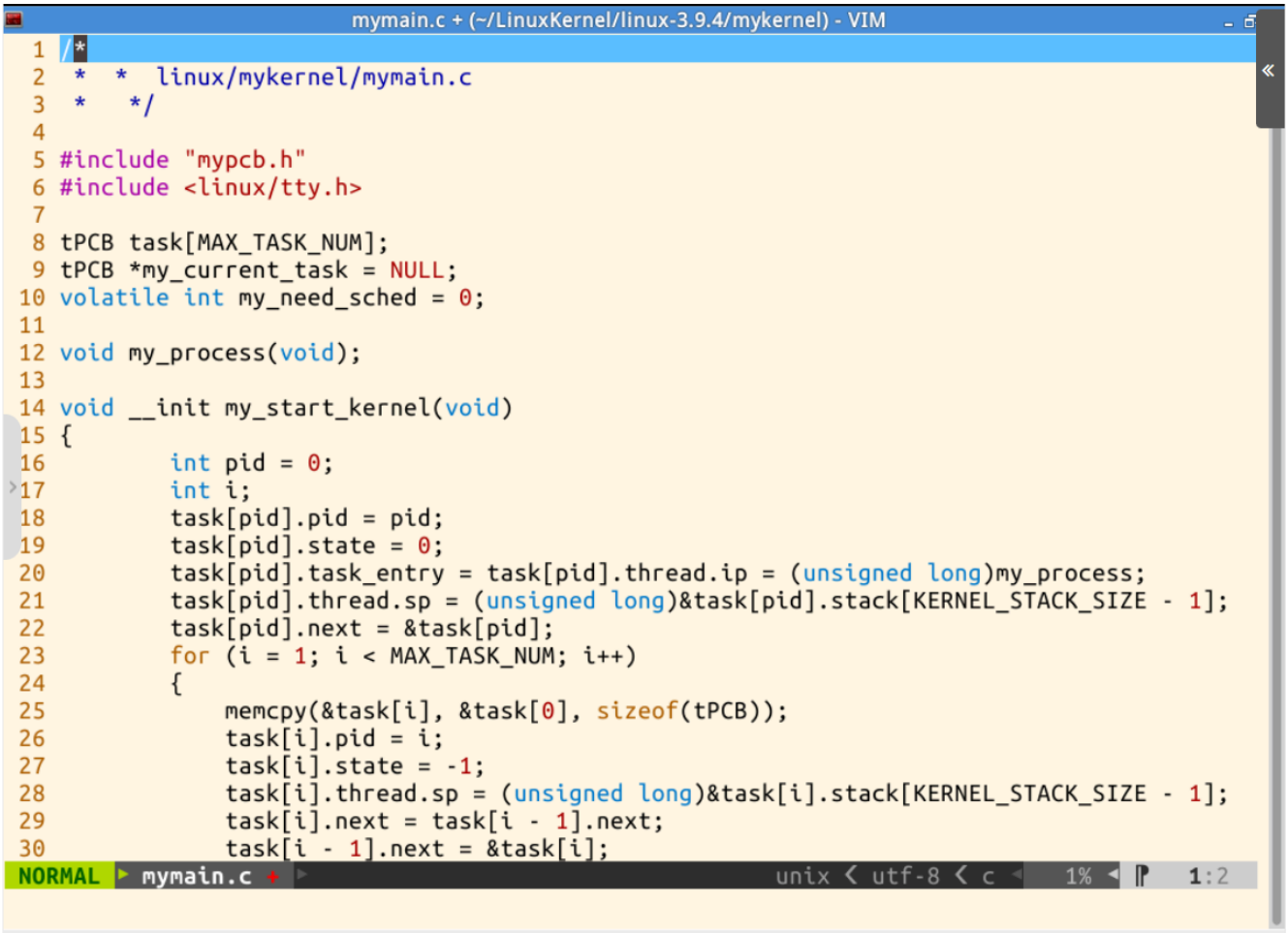
在 `~/LinuxKernel/linux-3.9.4/mykernel` 中创建 `mypcb.h`。截图展示了完整的 PCB 定义：



观察与分析：代码最多定义4个任务。`pid` 标识任务，`state` 表示本示例约定的状态；`thread.ip` 和 `thread.sp` 保存执行位置和栈指针；每个 PCB 中有独立的栈；`task_entry` 保存入口，`next` 用于组织任务链表。

3.6 修改 mymain.c：初始化任务与环形链表

在相同目录编辑 `mymain.c`。截图展示了任务数组、当前任务指针、调度标志，以及 `my_start_kernel()` 中初始化任务的部分代码。



截图可见代码先设置任务0的 PID、状态、入口和栈指针，再以它为模板初始化其他任务。其中两个连接语句可整理为：

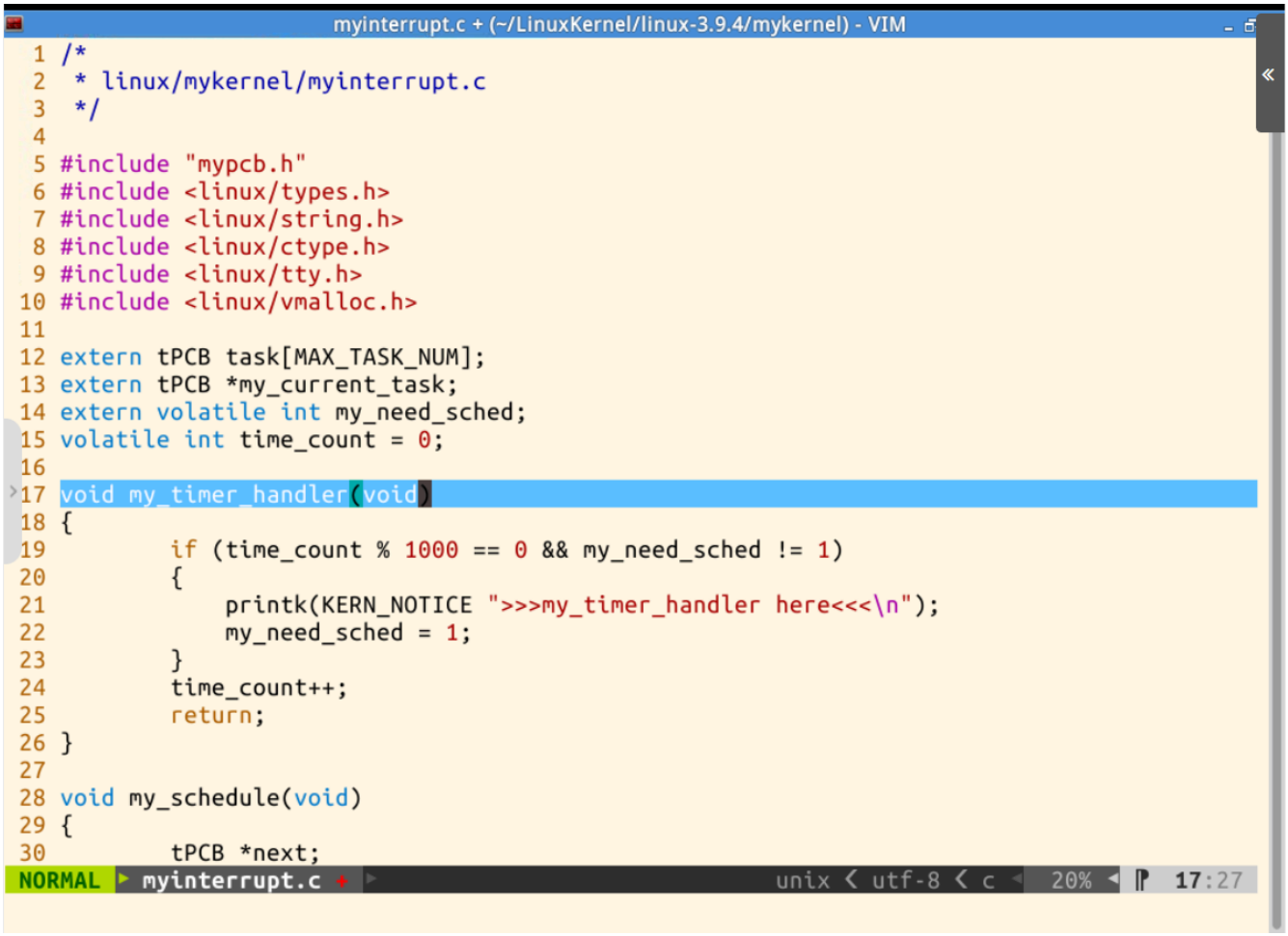
```
/* 初始化循环中的可见片段，并非完整文件 */
task[i].next = task[i - 1].next;
task[i - 1].next = &task[i];
```

观察与分析：任务0的 `next` 最初指向自身。依次插入任务1、2、3后，得到 `0→1→2→3→0` 的环形关系。每个新任务重新设置 PID 和自身栈指针，避免所有任务共用任务0的栈。截图中任务0的状态为0，其他新任务为-1；这些数值是此教学代码的约定。

3.7 修改 myinterrupt.c：由时钟提出调度请求

编辑 `myinterrupt.c`，增加调度相关代码。截图可见的时钟处理函数为：

```
void my_timer_handler(void)
{
    if (time_count % 1000 == 0 && my_need_sched != 1) {
        printk(KERN_NOTICE ">>>my_timer_handler here<<<\n");
        my_need_sched = 1;
    }
    time_count++;
    return;
}
```



观察与分析：`time_count` 记录处理次数，满足条件时设置 `my_need_sched=1`。当标志已经置位时，不重复提出同一个请求。由于计数器初值为0，第一次调用也满足取模条件。

这里的1000是计数间隔，不能直接解释为1000毫秒。换算实际时间需要时钟频率和处理路径等信息。截图只展示了 `my_schedule()` 的开头，未显示完整切换汇编；能直接确认的是时钟提出调度请求与调度函数入口已写入代码。

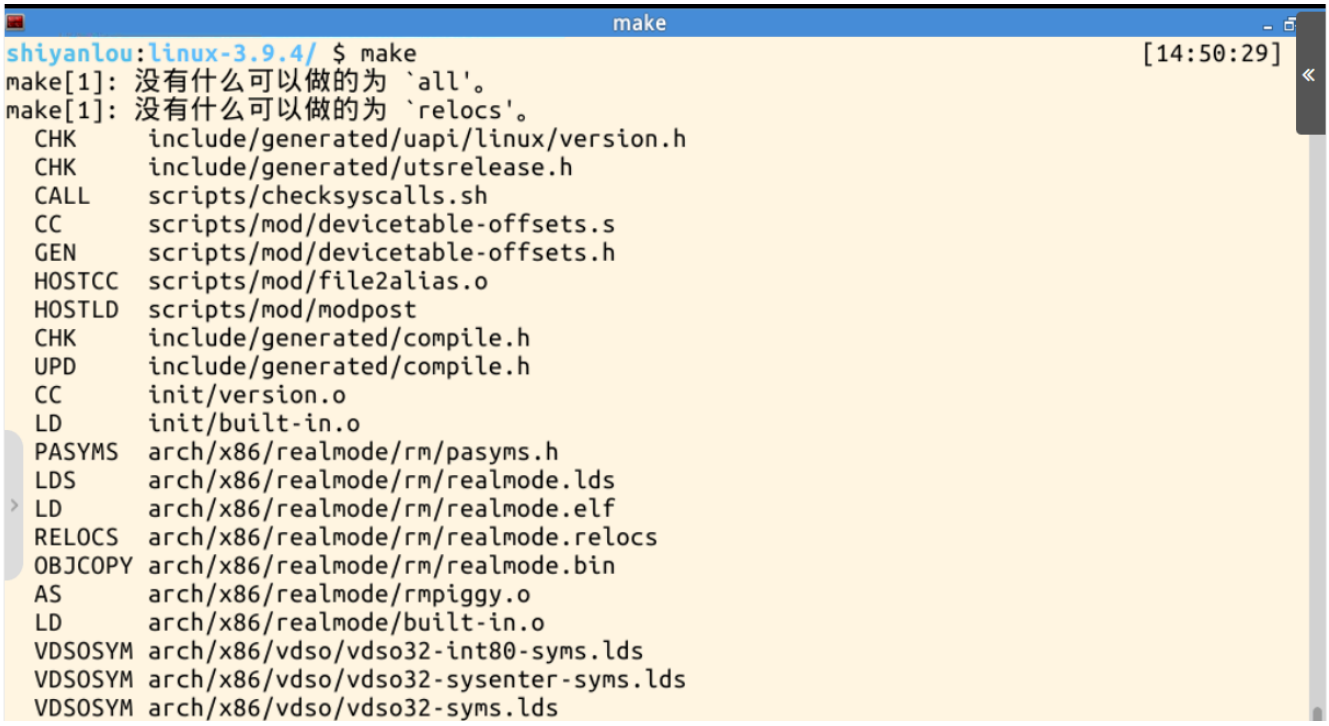
3.8 检查文件并重新编译

在 `mykernel/` 中使用 `ls` 检查文件，然后回到源码根目录重新编译：

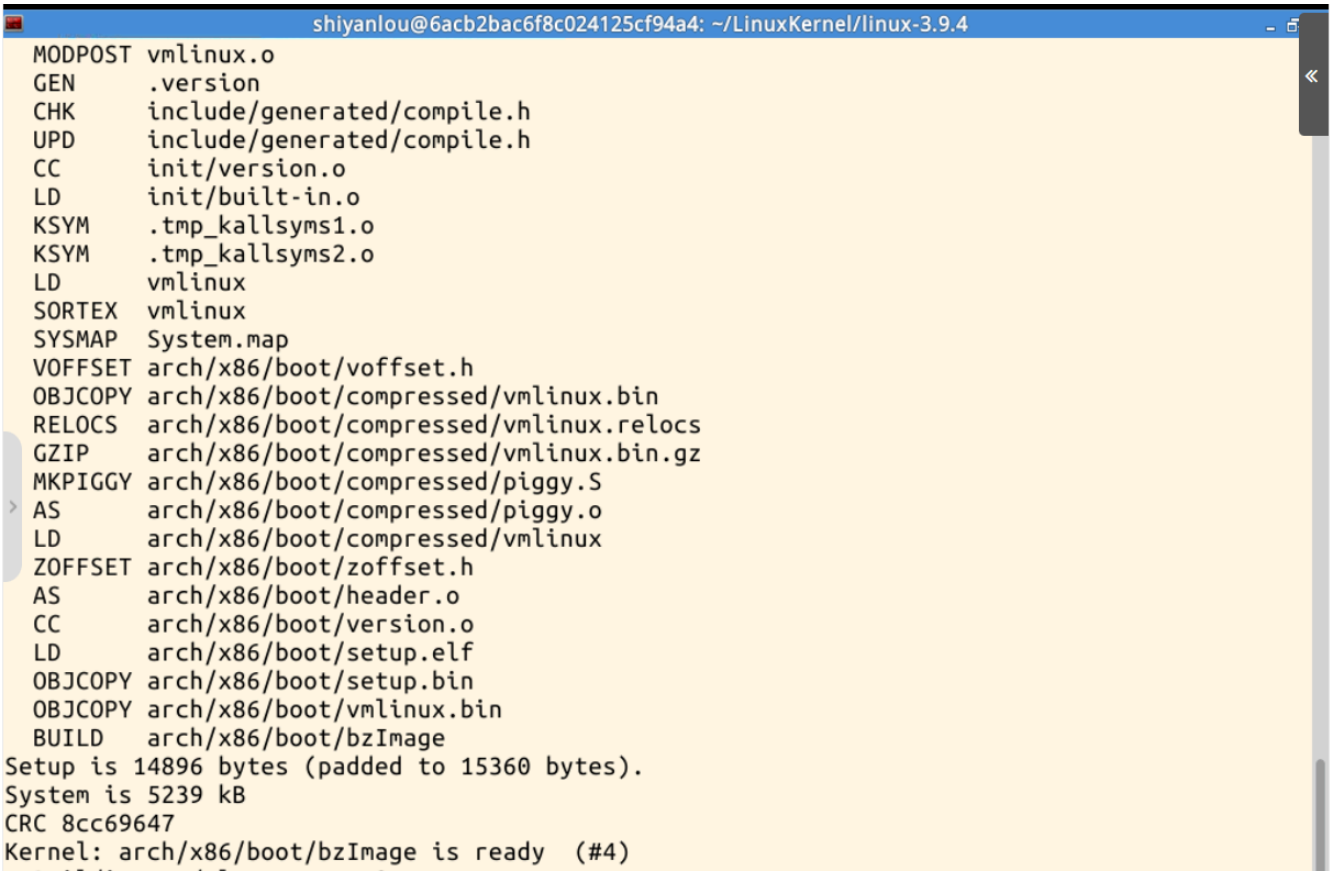
```
ls
cd ~/LinuxKernel/linux-3.9.4
make
```



图7可见 `mypcb.h`、`mymain.c`、`myinterrupt.c` 和 Makefile，同时存在 `.o` 等构建文件。



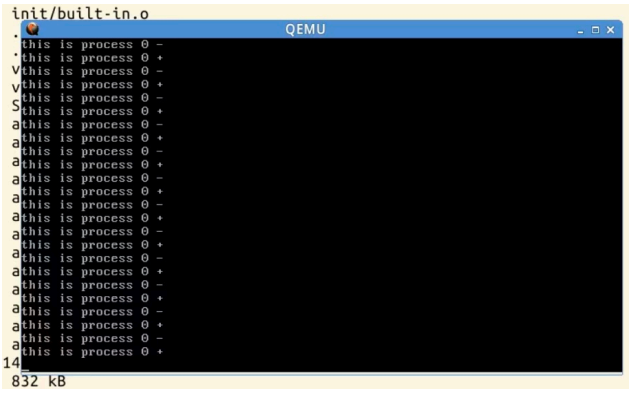
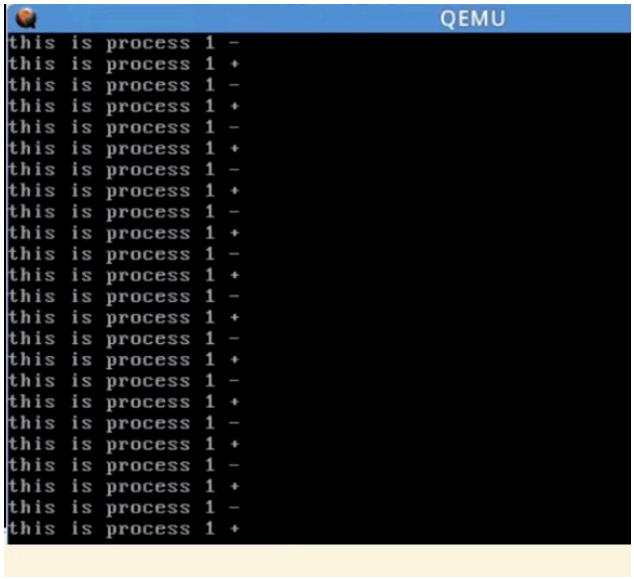
3.9 修改后内核编译完成



观察与分析：图9显示 `VMLINUX` 相关链接、符号和镜像构建阶段，最终出现 `Kernel: arch/x86/boot/bzImage is ready (#4)`，记录的 `System is 5239 kB`。这为修改阶段的镜像构建完成提供了证据。

3.10 运行结果：观察不同 PID

重新编译后，实验流程使用 QEMU 加载 `arch/x86/boot/bzImage`。以下为提供的两个运行画面：

图10：process 0运行输出	图11：process 1运行输出
	

观察与分析：图10反复出现 `this is process 0 -` 和 `this is process 0 +`；图11反复出现对应的 process 1 输出。与默认阶段相比，画面已经使用任务 PID 标记执行，记录中至少出现了两个不同任务。

3.11 核心机制：请求、选择与切换

根据可见的 PCB、链表和标志代码，简单轮转内核的原理可以概括为：

```
时钟处理：设置调度请求
    ↓
任务到达检查点：判断是否需要调度
    ↓
调度器：沿next选择下一个任务
    ↓
上下文切换：保存当前任务的执行状态，恢复下一个任务
    ↓
下一个任务继续执行
```

其中，“请求调度”“选中下一任务”和“真正切换上下文”是三个不同环节。上下文切换需要保存当前任务的恢复位置与栈，并切换到下一个任务的状态，否则多个任务即使拥有不同 PID，也无法分别从自己的位置继续执行。

补充说明：上述完整执行链是教学模型的原理解释。若任务只在主动检查点调用调度器，时钟负责置标志，那么切换会延迟到检查点，不应表述为时钟一到便立即强制抢占。截图没有展示完整进程函数和切换汇编，实际细节仍以最终源码为准。

04 | 易错点与注意事项

现有记录没有提供实际报错及排查日志，本节整理代码分析和材料复核中需要注意的概念，不虚构调试经历。

容易混淆的说法	本次理解与修正
无限循环就是故障	默认主循环和时钟都在执行；任务循环、空闲循环及服务线程都可能长期运行
时间片轮转能避免死锁	轮转分配 CPU 时间；死锁涉及资源等待关系，单靠轮转不能消除
轮转一定使程序更快	轮转有助于就绪任务获得运行机会，但切换有开销，本实验没有性能测量
有多个PID就等于同时执行	单 CPU 上通常是交替执行形成并发效果，不能据此证明并行
`volatile`可以保证同步	它不等于原子操作或完整同步机制；真实内核需考虑锁、屏障和并发条件
mykernel的PCB等同于真实进程描述符	教学PCB保留必要信息，真实Linux的 `task_struct` 及调度机制复杂得多
某个构建文件存在就说明本次成功	应检查本次结束提示、退出状态或日志，避免误用旧产物

公平性也有前提：任务应持续可运行、调度器能够继续推进。如果某个任务永远不进入必要的调度检查点，简化模型不能仅靠设置标志保证所有任务得到执行。缩短时间片可能改善响应，也可能增加切换开销，需要结合任务和实际测量判断。

05 | 本周总结

教材第三章让我从“查看单个函数”转向“沿启动路径理解内核”：源码目录帮助定位模块，配置决定实际构建内容，QEMU/GDB 将静态调用关系与运行过程联系起来。通过 `start_kernel()` 和 `rest_init()` 的分析，可以进一步区分0号空闲任务、1号 init 任务与2号 kthreadd 的来源和职责。

mykernel 实验把任务描述、独立栈、环形链表及调度请求集中到少量文件中。默认版本的主循环与时钟输出说明内核可以持续执行；修改阶段的编译和不同 PID 输出，则为理解多任务运行提供了具体观察。本次分析也明确了调度策略与上下文切换的区别，以及轮转、公平性、饥饿和死锁之间的边界。

后续需要结合完整源码进一步巩固首次启动与恢复执行的区别，并补充四个任务完整轮转的连续记录。阅读理解与实验现象相互验证，才能把“看到多个任务输出”推进到“解释任务为何能够切换并继续运行”。